

ABRA — Architecture Notes

Version 1.0 • Last updated 2026-07-23 • Will Hooper

The design decisions behind ABRA that aren't obvious from any single file: why the models are split across two languages, how the local app runs the real engines on your own machine, and how recency weighting keeps every model tracking the live metagame. Living document; updated with the code.

1. Why two languages (Python and JavaScript)

ABRA's engines are deliberately split. It is not an accident or inconsistency — each half of the system lives in the language that makes it shortest and safest to write.

The rule

- **JavaScript (Node) for anything that touches the battle engine.** CHOMP's damage calculator (`CHOMP/engine/champ-model.js`) is the validated, exact model of Champions damage — types, STAB, weather, items, abilities, the full 16-roll distribution. It is written in JS. Every model that needs *real game mechanics* reuses it directly instead of re-implementing (and re-validating) the damage formula:
- **MEDICHAM** (`engine/medicham.js`) — rollouts call CHOMP's `bestDamage` per hit.
- **KADABRA** (`engine/kadabra.js`) — the coach reconstructs scenes and cross-checks damage.
- **the set-builder, behaviour-clone, dynamics** — all Node, all sharing the same move/typing data. Re-porting the damage engine to Python would mean maintaining two copies of the most correctness-critical code in the project. That is the thing we most want to avoid.
- **Python (with numpy) for the statistical / machine-learning core.** JOLTEON's training (`engine/jolteon.py`) and DITTO's search (`engine/ditto.py`) are linear algebra: logistic-regression gradient descent, vectorised scoring of thousands of teams, rarity-aware regularisation, recency weighting. numpy expresses this in a few lines and runs it fast. Writing weighted logistic regression by hand in JS would be longer and slower with no upside.

The boundary, and why it barely matters to a user

The split falls on a clean seam: **mechanics = JS, learned numeric models = Python**. The one place they meet is DITTO, which uses JOLTEON (Python) as its fast evaluator and MEDICHAM (JS) as its grounded vetter — it shells out to both.

Crucially, **using the models needs almost no Python**:

- JOLTEON's *predictions* are a tiny logistic dot-product. We ship the trained weights as JSON and recompute the probability in pure JavaScript in the browser (`web/index.html`) and could do so in Node. You never need Python to *use* JOLTEON — only to *retrain* it.
- MEDICHAM, KADABRA, CHOMP are Node end to end.
- Only **DITTO** currently requires Python at run time (numpy for the team search). Because that is the sole run-time Python dependency, the planned **Node port of DITTO** removes Python from the live app entirely: JOLTEON weights are already portable, MEDICHAM is already JS, so a JS DITTO closes

the loop and the whole `localhost` app runs on Node alone. Retraining JOLTEON stays in Python, offline, where numpy earns its keep.

The Windows Python gotcha (documented because it will bite)

Windows ships a fake `python` command that opens the Microsoft Store instead of running Python. So on a machine without a real Python install, `python engine/ditto.py` prints *"Python was not found; run without arguments to install from the Microsoft Store."* The server (`server.js`) handles this: it probes `py`, then `python`, then `python3`, skips the Store stub, checks that numpy imports, and — if no real Python is found — returns a clear message telling you JOLTEON/MEDICHAM/KADABRA still work and how to install Python for DITTO (python.org, "Add to PATH", then `pip install numpy`). This probe is **lazy** (run on the first DITTO/JOLTEON request, not at startup) so the server always starts listening instantly.

2. The local app — running the real engines on your machine

`web/index.html` (mirrored to `app/index.html`, which the server prefers) is the ABRA WORLD site. Opened as a file, it runs **JOLTEON** live in the browser (embedded weights) — that alone needs no server. To run the *other* engines, `start.bat` launches `server.js`, a dependency-free Node HTTP server that:

- serves the site from `app/` (falling back to `web/`),
- exposes `/api/medicham`, `/api/kadabra` (Node engines), `/api/jolteon`, `/api/ditto` (Python engines), and `/api/stats` (live counts for the town),
- runs every computation **on your own machine** — nothing is sent anywhere.

`start.bat` checks Node is installed, opens the browser *after* a short delay (so the first load doesn't race the server), and keeps a window open as the running server. Everything is local; "server" here just means "a small program using your computer's own CPU," not a remote host.

3. Recency weighting — tracking a metagame that moves

The problem: concept drift

A competitive metagame is **non-stationary**. The teams that win shift week to week as players adapt, new cores emerge, and old ones fall off. A model trained with every game counted equally averages over that history, so it is always partly describing a metagame that no longer exists. In machine-learning terms this is **concept drift**, and the standard remedy is to weight recent data more heavily.

The mechanism: exponential decay with a half-life

Every training game gets a weight that decays exponentially with its age:

$$w_i = 0.5 ** (age_days_i / \tau)$$

where `age_days_i` is how old game *i* is relative to the newest game in the set, and **τ (tau) is the half-life in days**. Interpretation:

- a game from *today* has weight ~ 1 ,
- a game **one half-life** old counts **half**,
- **two half-lives** $\rightarrow$ a quarter, **three** $\rightarrow$ an eighth, and so on.

Weights are normalised to mean 1 so the regularisation scale (the rarity-aware L2) is unchanged; only the *relative* influence of recent vs. old games shifts. Setting $\tau \rightarrow \infty$ recovers the old equal-weight behaviour exactly, so decay is a strict generalisation, not a different model.

Where it plugs in

The weights enter the **weighted logistic-regression gradient** in JOLTEON's `fit()`:

$$\text{grad} = X^T (w \odot (p - y)) / \sum w + L2 \cdot \theta / \sum w$$

The same $w_i = 0.5^{(\text{age}/\tau)}$ applies identically to the **usage model** (weighted species counts) and the **behaviour-clone** (weighted move frequencies), so JOLTEON, DITTO's meta gauntlet, CHOMP's usage intel, and MEDICHAM's move priors all track the *current* ladder from one shared decay rule. The default half-life is **30 days** (`HALF_LIFE` env var); shorter τ reacts faster to a shifting meta but uses effectively less data, longer τ is steadier but laggier — a bias/variance knob.

Honest status on current data

ABRA's store currently spans ~2 days, so at $\tau = 30$ days every game's weight is ~1 and decay is a **no-op** — reported honestly (the equal-weight and recency-decayed rows are identical). The mechanism is unit-tested on synthetic 90-day data (oldest game weight 0.33, newest 2.14 at $\tau = 30d$) and will engage automatically as the corpus ages past a few weeks. It costs nothing now and means the models never need manual pruning of stale games later — the influence just fades on its own.

Why not simply delete old games?

Because rare-but-still-relevant information (an off-meta answer that occasionally matters) survives in old games, and a hard cutoff throws it away discontinuously. Exponential decay is a soft, continuous down-weighting: old games keep a small vote, recent games dominate, and nothing is ever lost from the store (which stays the durable, replayable record).

Related. [Simulator white paper](#) · [SLOWKING white paper](#) · [Cheat sheet](#) · [Changelog](#)